

```

/* =====
* Pioneer AI Service Framework V1.0.0
* 版权所有 © 2026 [秦晓望]
* 提交日期: 2026-05-04
* =====

// === 模块: apps/admin/src/middleware.ts ===
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('pi_auth_token')?.value;

  const protectedPaths = ['/dashboard', '/memberships', '/orders', '/payments',

  const isProtectedPath = protectedPaths.some((path) =>
    request.nextUrl.pathname === path || request.nextUrl.pathname.startsWith(`$
  );

  if (isProtectedPath && !token) {
    // According to req: If unauthenticated in /dashboard, /memberships, etc.,
    // Assuming the login page is physically at /login in the admin app, or we
    // Given standard NextJS architectures, navigating to /login is appropriate
    // If Admin app doesn't have a /login page natively, standard would rely on
    const loginUrl = new URL('/login', request.url);
    loginUrl.searchParams.set('mode', 'signin');
    return NextResponse.redirect(loginUrl);
  }

  // Prevent redirect to signup if somehow triggered to root mapping
  if (request.nextUrl.pathname === '/') {
    return NextResponse.next(); // Explicit approve
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/(?!api|_next/static|_next/image|favicon.ico).*'],
};

// === 模块: apps/web/src/middleware.ts ===
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  // Check for the presence of the Pi auth token
  const token = request.cookies.get('pi_auth_token')?.value;

```

```

// Paths that require authentication
const protectedPaths = ['/dashboard', '/account', '/billing'];

const isProtectedPath = protectedPaths.some((path) =>
  request.nextUrl.pathname === path || request.nextUrl.pathname.startsWith(`${
});

// Redirect to login (Sign In) if trying to access a protected route without
if (isProtectedPath && !token) {
  const loginUrl = new URL('/login', request.url);
  // Add mode=signin just as an explicit UX/UI hint if needed by components
  loginUrl.searchParams.set('mode', 'signin');
  return NextResponse.redirect(loginUrl);
}

// Ensure the landing page / is ALWAYS accessible to unauthenticated users.
// Next.js allows it unless explicitly intercepted, but this is added for cla
// to satisfy strict requirements forbidding auto-redirects to signup on /.
if (request.nextUrl.pathname === '/') {
  return NextResponse.next();
}

return NextResponse.next();
}

export const config = {
  // Apply middleware to all routes except api, _next static assets, and favico
  matcher: ['/(?!api|_next/static|_next/image|favicon.ico).*'],
};

// === 模块: apps/web/src/app/api/auth/pi/route.ts ===
import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';
import { cookies } from 'next/headers';
import { signSessionToken } from '@/lib/session';

const prisma = new PrismaClient();

// Pi Platform API 基础地址
const PI_API_BASE = process.env.PI_PLATFORM_API_BASE ?? 'https://[YOUR_DOMAIN]'

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const { accessToken, piUid, username, merchantId } = body;

    if (!accessToken || !piUid || !username || !merchantId) {
      return NextResponse.json(
        { success: false, error: 'Missing parameters: accessToken, piUid, usern

```

```

    { status: 400 }
  );
}

// — 步骤 1: 用 accessToken 调用 Pi Platform API 验证真实身份 —————
// GET /v2/me 使用 Authorization: Bearer <accessToken>
// 这是 Pi 官方推荐的 token 验证方式, 无需 PI_API_KEY
let verifiedUid: string;
let verifiedUsername: string;

try {
  const piMeRes = await fetch(`${PI_API_BASE}/v2/me`, {
    headers: {
      Authorization: `Bearer ${accessToken}`,
    },
  });

  if (!piMeRes.ok) {
    const errBody = await piMeRes.text();
    // [DEBUG_REMOVED]
    return NextResponse.json(
      { success: false, error: `Pi Platform 验证失败 (${piMeRes.status}): to`,
        { status: 401 }
      );
  }

  const piUser = await piMeRes.json();
  verifiedUid = piUser.uid;
  verifiedUsername = piUser.username;

  // 防止前端伪造 uid
  if (verifiedUid !== piUid) {
    // [DEBUG_REMOVED]
    return NextResponse.json(
      { success: false, error: '身份验证失败: uid 不匹配' },
      { status: 403 }
    );
  }
} catch (fetchErr) {
  // [DEBUG_REMOVED]
  return NextResponse.json(
    { success: false, error: 'Pi Platform API 网络不可达, 请检查服务器网络配置',
      { status: 502 }
    );
}

// — 步骤 2: Upsert Customer 记录 —————
const customer = await prisma.customer.upsert({
  where: {
    merchantId_piUid: {

```

```

        merchantId: merchantId,
        piUid: verifiedUid,
      },
    },
    update: {
      username: verifiedUsername,
    },
    create: {
      merchantId: merchantId,
      piUid: verifiedUid,
      username: verifiedUsername,
    },
  },
});

```

// — 步骤 3: 签发 HMAC session token 写入 HttpOnly Cookie

```

const secureToken = signSessionToken(verifiedUid);
const cookieStore = cookies();
cookieStore.set('pi_auth_token', secureToken, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  path: '/',
  maxAge: 60 * 60 * 24 * 7, // 7 天
});

```

// [DEBUG_REMOVED]

```

return NextResponse.json({
  success: true,
  user: { uid: verifiedUid, username: verifiedUsername },
  token: secureToken, // 发送给前端, 作为 localStorage 备用
});

```

```

} catch (error: any) {

```

// [DEBUG_REMOVED]

```

  return NextResponse.json({ success: false, error: error.message }, { status
}

```

```

}

```

// === 模块: apps/web/src/app/api/payments/approve/route.ts ===

```

import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';

```

```

const prisma = new PrismaClient();

```

```

export async function POST(req: Request) {
  try {

```

```

    const body = await req.json();
    const { paymentId, orderId } = body;

```

```

if (!paymentId) {
  return NextResponse.json({ success: false, error: 'Missing paymentId' },
  }

// 1. 幂等策略：检查数据库状态
const existingPayment = await prisma.payment.findUnique({
  where: { piPaymentId: paymentId }
});

if (!existingPayment) {
  const order = await prisma.order.findUnique({ where: { orderNo: orderId }
  if (order) {
    await prisma.payment.create({
      data: {
        orderId: order.id,
        piPaymentId: paymentId,
        amount: order.amount,
        status: 'PENDING',
        developerApproved: true,
        approvedAt: new Date(),
        memo: `Paying for order ${order.orderNo}`
      }
    });
    await prisma.order.update({
      where: { id: order.id },
      data: { status: 'PENDING_APPROVAL', paymentId: paymentId }
    });
  }
}

// 2. 极其关键：必须向 Pi 官方服务器发送 Approve 请求，否则 SDK 会死锁！
const piApiBase = process.env.PI_PLATFORM_API_BASE || 'https://[YOUR_DOMAIN
const apiKey = process.env.PI_API_KEY /* [REDACTED_环境配置] */;

if (!apiKey) {
  // [DEBUG_REMOVED]
  return NextResponse.json({ success: false, error: 'Missing PI_API_KEY' },
  }

const piRes = await fetch(`${piApiBase}/v2/payments/${paymentId}/approve`,
  method: 'POST',
  headers: {
    'Authorization': `Key ${apiKey}`
  }
});

if (!piRes.ok) {
  const errText = await piRes.text();
  // [DEBUG_REMOVED]

```

```

    // 如果 Pi 报错说已经 approve 过了, 可以放行
    if (!errText.includes('already approved')) {
        return NextResponse.json({ success: false, error: `Pi API Error: ${errT
    }
    }

    return NextResponse.json({ success: true });
} catch (error: any) {
    // [DEBUG_REMOVED]
    return NextResponse.json({ success: false, error: error.message }, { status
}
}

// === 模块: apps/web/src/app/api/payments/complete/route.ts ===
import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

export async function POST(req: Request) {
    try {
        const body = await req.json();
        const { paymentId, txid } = body;

        if (!paymentId || !txid) {
            return NextResponse.json({ success: false, error: 'Missing parameters' },
        }

        const payment = await prisma.payment.findUnique({
            where: { piPaymentId: paymentId },
            include: { order: true }
        });

        // 1. 极其关键: 向 Pi 官方服务器发送 Complete 请求, 真正完结这笔支付
        const piApiBase = process.env.PI_PLATFORM_API_BASE || 'https://[YOUR_DOMAIN
        const apiKey = process.env.PI_API_KEY /* [REDACTED_环境配置] */;

        if (!apiKey) {
            // [DEBUG_REMOVED]
            return NextResponse.json({ success: false, error: 'Missing PI_API_KEY' },
        }

        const piRes = await fetch(`${piApiBase}/v2/payments/${paymentId}/complete`,
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Key ${apiKey}`
            },
            body: JSON.stringify({ txid })

```

```

});

if (!piRes.ok) {
  const errText = await piRes.text();
  // [DEBUG_REMOVED]
  if (!errText.includes('already completed')) {
    return NextResponse.json({ success: false, error: `Pi API Error: ${errT
  }
}

if (!payment || payment.status === 'COMPLETED') {
  // 幂等边界
  return NextResponse.json({ success: true, message: 'Already completed l
}

// 2. 更新本地数据库
await prisma.$transaction(async (tx) => {
  await tx.payment.update({
    where: { piPaymentId: paymentId },
    data: {
      txid: txid,
      status: 'COMPLETED',
      transactionVerified: true,
      developerCompleted: true,
      completedAt: new Date()
    }
  });
});

const updatedOrder = await tx.order.update({
  where: { id: payment.orderId },
  data: { status: 'COMPLETED' }
});

const durationDays = 30;
let dbMembership = await tx.membership.findFirst({
  where: { merchantId: updatedOrder.merchantId }
});

if (!dbMembership) {
  dbMembership = await tx.membership.create({
    data: {
      merchantId: updatedOrder.merchantId,
      name: 'AI Framework Subscription',
      mode: 'TIME_BASED',
      price: 0,
    }
  });
});

const endAt = new Date();

```

```

        endAt.setDate(endAt.getDate() + durationDays);

        await tx.customerMembership.create({
            data: {
                customerId: updatedOrder.customerId,
                membershipId: dbMembership.id,
                startAt: new Date(),
                endAt: endAt,
                status: 'ACTIVE'
            }
        });
    });

    return NextResponse.json({ success: true });
} catch (error: any) {
    // [DEBUG_REMOVED]
    return NextResponse.json({ success: false, error: error.message }, { status
}

// === 模块: prisma/schema.prisma ===
generator client {
    provider = "prisma-client-js"
}

datasource db {
    provider = "postgresql"
    url      = env("DATABASE_URL")
}

enum MerchantStatus {
    ACTIVE
    INACTIVE
    SUSPENDED
}

enum MerchantType {
    BEAUTY
    FITNESS
    EDUCATION
    CONSULTING
    REPAIR
    RETAIL
    GENERIC
}

enum UserRole {
    OWNER
    STAFF

```



```
VIEWER
}

enum UserStatus {
    ACTIVE
    INACTIVE
    BANNED
}

enum ServiceType {
    SERVICE
    PRODUCT
    PACKAGE
}

enum ServiceStatus {
    ACTIVE
    INACTIVE
    DRAFT
}

enum OrderStatus {
    DRAFT
    PENDING_PAYMENT
    PENDING_APPROVAL
    APPROVED
    COMPLETED
    FAILED
    CANCELLED
    REFUNDED
}

enum PaymentStatus {
    PENDING
    APPROVED
    COMPLETED
    FAILED
    CANCELLED
    REFUNDED
}

enum BookingStatus {
    PENDING
    CONFIRMED
    COMPLETED
    CANCELLED
    NO_SHOW
}

enum MembershipMode {
```

```

    TIME_BASED
    USAGE_BASED
    UNLIMITED
}

enum MembershipStatus {
    ACTIVE
    EXPIRED
    CANCELLED
}

enum CheckoutMode {
    SINGLE
    SUBSCRIPTION
}

model Merchant {
    id          String          @id @default(cuid())
    name        String
    type        MerchantType    @default(GENERIC)
    status      MerchantStatus  @default(ACTIVE)
    logo        String?
    contactName String?         @map("contact_name")
    contactPhone String?        @map("contact_phone")
    themeConfig Json?           @map("theme_config")
    createdAt   DateTime         @default(now()) @map("created_at")
    updatedAt   DateTime         @updatedAt @map("updated_at")

    merchantUsers MerchantUser[]
    customers      Customer[]
    services       Service[]
    orders         Order[]
    bookings       Booking[]
    memberships    Membership[]
    appConfig      AppConfig?

    @@map("merchants")
}

model MerchantUser {
    id          String          @id @default(cuid())
    merchantId  String          @map("merchant_id")
    piUid       String          @map("pi_uid")
    username    String
    role        UserRole        @default(STAFF)
    status      UserStatus      @default(ACTIVE)
    createdAt   DateTime         @default(now()) @map("created_at")

    merchant Merchant @relation(fields: [merchantId], references: [id], onDelete:

```

```

    @@index([merchantId])
    @@index([piUid])
    @@map("merchant_users")
}

model Customer {
    id                String      @id @default(cuid())
    merchantId        String      @map("merchant_id")
    piUid             String      @map("pi_uid")
    username          String
    displayName        String?    @map("display_name")
    membershipStatus  MembershipStatus? @map("membership_status")
    createdAt         DateTime    @default(now()) @map("created_at")

    merchant          Merchant    @relation(fields: [merchantId], refer
    orders            Order[]
    bookings          Booking[]
    customerMemberships CustomerMembership[]

    @@index([merchantId])
    @@index([piUid])
    @@unique([merchantId, piUid])
    @@map("customers")
}

model Service {
    id                String      @id @default(cuid())
    merchantId        String      @map("merchant_id")
    type              ServiceType @default(SERVICE)
    title             String
    description        String?
    price             Decimal      @db.Decimal(10, 2)
    durationMinutes    Int?        @map("duration_minutes")
    status            ServiceStatus @default(ACTIVE)
    createdAt         DateTime    @default(now()) @map("created_at")

    merchant Merchant @relation(fields: [merchantId], references: [id], onDelete
    orders  Order[]
    bookings Booking[]

    @@index([merchantId])
    @@index([merchantId, status])
    @@map("services")
}

model Order {
    id                String      @id @default(cuid())
    merchantId        String      @map("merchant_id")
    customerId        String      @map("customer_id")
    serviceId         String?     @map("service_id")

```

```

orderNo      String      @unique @map("order_no")
amount       Decimal     @db.Decimal(10, 2)
currency     String      @default("PI")
status       OrderStatus @default(DRAFT)
paymentId    String?     @map("payment_id")
note         String?
createdAt    DateTime     @default(now()) @map("created_at")
updatedAt    DateTime     @updatedAt @map("updated_at")

merchant     Merchant    @relation(fields: [merchantId], references: [id])
customer     Customer    @relation(fields: [customerId], references: [id])
service      Service?    @relation(fields: [serviceId], references: [id])
payments     Payment[]

@@index([merchantId])
@@index([customerId])
@@index([status])
@@index([merchantId, status])
@@map("orders")
}

model Payment {
  id          String      @id @default(cuid())
  orderId     String      @map("order_id")
  piPaymentId String      @unique @map("pi_payment_id")
  txid        String?     @unique
  amount      Decimal     @db.Decimal(10, 2)
  memo        String?
  status      PaymentStatus @default(PENDING)
  developerApproved Boolean @default(false) @map("developer_approved")
  transactionVerified Boolean @default(false) @map("transaction_verified")
  developerCompleted Boolean @default(false) @map("developer_completed")
  userCancelled Boolean @default(false) @map("user_cancelled")
  createdAt   DateTime     @default(now()) @map("created_at")
  approvedAt  DateTime?    @map("approved_at")
  completedAt DateTime?    @map("completed_at")

  order Order @relation(fields: [orderId], references: [id])

  @@index([orderId])
  @@index([status])
  @@index([piPaymentId])
  @@map("payments")
}

model Booking {
  id          String      @id @default(cuid())
  merchantId  String      @map("merchant_id")
  customerId  String      @map("customer_id")
  serviceId   String?     @map("service_id")

```

```

    slotStart    DateTime          @map("slot_start")
    slotEnd      DateTime          @map("slot_end")
    status       BookingStatus     @default(PENDING)
    note         String?
    createdAt    DateTime          @default(now()) @map("created_at")

    merchant Merchant @relation(fields: [merchantId], references: [id])
    customer Customer @relation(fields: [customerId], references: [id])
    service      Service? @relation(fields: [serviceId], references: [id])

    @@index([merchantId])
    @@index([customerId])
    @@index([merchantId, slotStart])
    @@index([status])
    @@map("bookings")
}

model Membership {
    id            String            @id @default(cuid())
    merchantId    String            @map("merchant_id")
    name          String
    mode          MembershipMode
    price         Decimal           @db.Decimal(10, 2)
    validDays     Int?              @map("valid_days")
    totalUses     Int?              @map("total_uses")
    benefitsJson  Json?             @map("benefits_json")
    status        ServiceStatus     @default(ACTIVE)
    createdAt     DateTime          @default(now()) @map("created_at")

    merchant      Merchant          @relation(fields: [merchantId], refe
    customerMemberships CustomerMembership[]

    @@index([merchantId])
    @@index([merchantId, status])
    @@map("memberships")
}

model CustomerMembership {
    id            String            @id @default(cuid())
    customerId    String            @map("customer_id")
    membershipId  String            @map("membership_id")
    startAt       DateTime          @map("start_at")
    endAt         DateTime?         @map("end_at")
    remainingUses Int?              @map("remaining_uses")
    status        MembershipStatus @default(ACTIVE)
    createdAt     DateTime          @default(now()) @map("created_at")

    customer      Customer          @relation(fields: [customerId], references: [id], onDel
    membership    Membership @relation(fields: [membershipId], references: [id])

```

```

    @@index([customerId])
    @@index([membershipId])
    @@index([customerId, status])
    @@map("customer_memberships")
  }

model AppConfig {
  id                String          @id @default(cuid())
  merchantId        String          @unique @map("merchant_id")
  modulesEnabled    Json            @map("modules_enabled")
  homepageLayout    String          @default("service-first") @map("homepage_layout")
  industrySkin      String          @default("generic") @map("industry_skin")
  checkoutMode      CheckoutMode   @default(SINGLE) @map("checkout_mode")
  enableCoupon      Boolean         @default(false) @map("enable_coupon")
  enableA2uReward   Boolean         @default(false) @map("enable_a2u_reward")
  createdAt         DateTime        @default(now()) @map("created_at")
  updatedAt         DateTime        @updatedAt @map("updated_at")

  merchant Merchant @relation(fields: [merchantId], references: [id], onDelete:

  @@map("app_configs")
}

// === 模块: apps/admin/src/app/orders/page.tsx ===
'use client';

// 后台 - 订单管理

import { Suspense, useEffect, useState } from 'react';
import { useRouter, useSearchParams, usePathname } from 'next/navigation';

interface Order {
  id: string;
  orderNo: string;
  amount: number;
  status: string;
  createdAt: string;
  customer?: { username: string };
  service?: { title: string };
}

interface Pagination {
  page: number;
  limit: number;
  total: number;
  totalPages: number;
}

const STATUS_OPTIONS = ['ALL', 'PENDING PAYMENT', 'APPROVED', 'COMPLETED', 'CAN

```

```

function OrdersContent() {
  const router = useRouter();
  const pathname = usePathname();
  const searchParams = useSearchParams();

  const currentStatus = searchParams.get('status') || 'ALL';
  const currentPage = parseInt(searchParams.get('page') || '1', 10);

  const [orders, setOrders] = useState<Order[]>([]);
  const [pagination, setPagination] = useState<Pagination | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);
    const qs = new URLSearchParams();
    if (currentStatus !== 'ALL') qs.set('status', currentStatus);
    qs.set('page', currentPage.toString());
    qs.set('limit', '10');

    fetch(`/api/admin/orders?${qs.toString()}`, { credentials: 'include' })
      .then((r) => r.json())
      .then((data: { orders: Order[], pagination: Pagination }) => {
        setOrders(data.orders ?? []);
        setPagination(data.pagination ?? null);
        setLoading(false);
      })
      .catch(() => setLoading(false));
  }, [currentStatus, currentPage]);

  const handleStatusChange = (status: string) => {
    const params = new URLSearchParams(searchParams.toString());
    if (status === 'ALL') {
      params.delete('status');
    } else {
      params.set('status', status);
    }
    // Changing status resets page to 1
    params.set('page', '1');
    router.push(`${pathname}?${params.toString()}`);
  };

  const handlePageChange = (newPage: number) => {
    if (newPage < 1 || (pagination && newPage > pagination.totalPages)) return;
    const params = new URLSearchParams(searchParams.toString());
    params.set('page', newPage.toString());
    router.push(`${pathname}?${params.toString()}`);
  };

  return (

```

```

<div>
  <h1 className="text-2xl font-bold text-gray-800 mb-6">订单管理</h1>

  {/* 状态筛选 */}
  <div className="flex flex-wrap gap-2 mb-6">
    {STATUS_OPTIONS.map((s) => (
      <button
        key={s}
        onClick={() => handleStatusChange(s)}
        className={`px-4 py-2 rounded-xl text-sm font-medium transition-colors
          ${currentStatus === s ? 'bg-[#7C3AED] text-white' : 'bg-gray-100 text-gray-800'}
        `}
      >
        {s === 'ALL' ? '全部' : s}
      </button>
    ))}
  </div>

  {/* 订单表格 */}
  <div className="bg-white rounded-2xl overflow-hidden shadow-sm border border-gray-100">
    <table className="w-full text-sm">
      <thead className="bg-gray-50 border-b">
        <tr>
          <th key={h} className="text-left px-4 py-3 text-gray-500 font-medium">{h}
        </th>
      </tr>
    </thead>
    <tbody>
      {loading ? (
        <tr>
          <td colspan={6} className="text-center py-10 text-gray-300">加载中...
        </td>
      </tr>
      ) : orders.length === 0 ? (
        <tr>
          <td colspan={6} className="text-center py-10 text-gray-400">暂无订单
        </td>
      </tr>
      ) : (
        orders.map((order) => (
          <tr key={order.id} className="border-b last:border-0 hover:bg-gray-50">
            <td className="px-4 py-3 font-mono text-xs text-gray-600">{order.id}
            <td className="px-4 py-3 text-gray-700">{order.customer?.user?.name}
            <td className="px-4 py-3 text-gray-700">{order.service?.title}
            <td className="px-4 py-3 text-[#7C3AED] font-semibold">¥ {Number(order.amount).toLocaleString('zh-CN')}
            <td className="px-4 py-3">
              <span className="bg-gray-100 text-gray-600 text-xs px-2 py-1 rounded">{order.status}</span>
            <td className="px-4 py-3 text-gray-400 text-xs">
              {new Date(order.createdAt).toLocaleString('zh-CN')}
            </td>
          </tr>
        ))
      )
    </tbody>
  </table>

```



```

        </tr>
      ))
    })
  </tbody>
</table>
</div>

{/* 分页器 */}
{pagination && pagination.totalPages > 1 && (
  <div className="flex items-center justify-between text-sm text-gray-600"
    <div>
      共 {pagination.total} 条记录, 当前第 {pagination.page} / {pagination.t
    </div>
    <div className="flex items-center gap-2">
      <button
        onClick={() => handlePageChange(pagination.page - 1)}
        disabled={pagination.page <= 1}
        className="px-3 py-1.5 rounded-lg border border-gray-200 hover:bg
      >
        上一页
      </button>
      <button
        onClick={() => handlePageChange(pagination.page + 1)}
        disabled={pagination.page >= pagination.totalPages}
        className="px-3 py-1.5 rounded-lg border border-gray-200 hover:bg
      >
        下一页
      </button>
    </div>
  </div>
)}
</div>
);
}

export default function AdminOrdersPage() {
  return (
    <Suspense fallback={<div className="p-10 text-center text-gray-500">Loading
      <OrdersContent />
    </Suspense>
  );
}

```

```

// === 模块: apps/admin/src/app/memberships/page.tsx ===
'use client';

```

```
// 后台 - 会员方案管理
```

```
import { useEffect, useState } from 'react';

interface Membership {
  id: string;
  name: string;
  mode: string;
  price: number;
  validDays: number | null;
  totalUses: number | null;
  status: string;
}

export default function AdminMembershipsPage() {
  const [memberships, setMemberships] = useState<Membership[]>([]);
  const [loading, setLoading] = useState(true);
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [creating, setCreating] = useState(false);
  const [errorMsg, setErrorMsg] = useState('');

  // Form states
  const [formData, setFormData] = useState({
    name: '',
    mode: 'TIME_BASED',
    price: '',
    validDays: '',
    totalUses: ''
  });

  const fetchMemberships = () => {
    setLoading(true);
    fetch('/api/admin/memberships', { credentials: 'include' })
      .then((r) => r.json())
      .then((data: { memberships: Membership[] }) => {
        setMemberships(data.memberships ?? []);
        setLoading(false);
      })
      .catch(() => setLoading(false));
  };

  useEffect(() => {
    fetchMemberships();
  }, []);

  const handleCreate = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrorMsg('');
    setCreating(true);

    try {
      const res = await fetch('/api/admin/memberships', {
```

```

    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      name: formData.name,
      mode: formData.mode,
      price: parseFloat(formData.price),
      validDays: formData.validDays ? parseInt(formData.validDays) : null,
      totalUses: formData.totalUses ? parseInt(formData.totalUses) : null,
    }),
    credentials: 'include'
  });
  const result = await res.json();
  if (!res.ok || !result.success) {
    throw new Error(result.error || '创建失败');
  }
  // Optimistic or real refresh
  fetchMemberships();
  setIsModalOpen(false);
  setFormData({ name: '', mode: 'TIME_BASED', price: '', validDays: '', tot
} catch (err: any) {
  setErrMsg(err.message);
} finally {
  setCreating(false);
}
};

return (
  <div className="relative">
    <div className="flex items-center justify-between mb-6">
      <h1 className="text-2xl font-bold text-gray-800">会员方案</h1>
      <button
        onClick={() => setIsModalOpen(true)}
        className="bg-[#7C3AED] hover:bg-[#6D28D9] text-white px-4 py-2 round
      >
        + 新增方案
      </button>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
      {loading ? (
        [1, 2, 3].map((i) => <div key={i} className="bg-white rounded-2xl p-5
      ) : memberships.length === 0 ? (
        <div className="col-span-full text-center py-20 text-gray-400">
          <div className="text-5xl mb-4">★</div>
          <p>暂无会员方案</p>
        </div>
      ) : (
        memberships.map((m) => (
          <div key={m.id} className="bg-white rounded-2xl p-5 shadow-sm borde
            <div className="flex justify-between items-start mb-2">

```

```

        <h3 className="font-bold text-gray-800 text-lg">{m.name}</h3>
        <span className={`text-xs px-2 py-1 rounded-lg ${
            m.status === 'ACTIVE' ? 'bg-green-100 text-green-700' : 'bg-g
        }`} >{m.status === 'ACTIVE' ? '生效中' : '已下架'}</span>
    </div>
    <p className="text-[#7C3AED] font-bold text-2xl mb-4">n {Number(n

    <div className="text-sm text-gray-500 mt-auto mb-4 bg-gray-50 p-3
        <p>模式: {m.mode === 'TIME_BASED' ? '有效时长卡' : m.mode === 'US
        {m.validDays && <p>额度: 有效期 {m.validDays} 天</p>}
        {m.totalUses && <p>额度: 共 {m.totalUses} 次</p>}
    </div>

    {/* Not connected to backend yet, but UI required us to not remov
        User said "不允许新增"看起来能点但其实没接后端"的假按钮", so we disa
    <div className="flex gap-2 border-t pt-3">
        <button disabled title="编辑功能尚在排期中" className="flex-1 text
            编辑
        </button>
        <button disabled title="下架功能尚在排期中" className="flex-1 text
            下架
        </button>
    </div>
</div>
    ))
    )}
</div>

{isModalOpen && (
    <div className="fixed inset-0 bg-black/50 flex items-center justify-cen
        <div className="bg-white rounded-2xl p-6 w-full max-w-md shadow-2xl">
            <h2 className="text-xl font-bold mb-4 text-gray-800">新增会员方案</h2

            {errorMsg && <div className="mb-4 text-sm text-red-600 bg-red-50 bc

        <form onSubmit={handleCreate} className="space-y-4">
            <div>
                <label className="block text-sm font-medium text-gray-700 mb-1"
                <input required type="text" value={formData.name} onChange={e =
            </div>

            <div>
                <label className="block text-sm font-medium text-gray-700 mb-1"
                <input required type="number" step="0.01" min="0" value={formDa
            </div>

            <div>
                <label className="block text-sm font-medium text-gray-700 mb-1"
                <select value={formData.mode} onChange={e => setFormData({...fc
                    <option value="TIME_BASED">有效时长卡 (如:月卡)</option>

```

```

        <option value="USAGE_BASED">使用次数卡 (如:10次卡)</option>
        <option value="UNLIMITED">不限量/永久订阅</option>
    </select>
</div>

{ (formData.mode === 'TIME_BASED' || formData.mode === 'USAGE_BASED'
  <div>
    <label className="block text-sm font-medium text-gray-700 mb-2">
      {formData.mode === 'TIME_BASED' ? '有效天数' : '包含次数'}
    </label>
    <input required type="number" min="1" value={formData.mode === 'TIME_BASED' ? formData.validDays : formData.usageCount} />
  </div>
)}

<div className="flex justify-end gap-3 pt-4 border-t mt-6">
  <button type="button" onClick={() => setIsModalOpen(false)} className="text-gray-500">取消</button>
  <button type="submit" disabled={creating} className="px-4 py-2 text-white bg-blue-600">{creating ? '提交...' : '确认创建'}</button>
</div>
</form>
</div>
</div>
)}
</div>
);
}

```

```

// === 模块: apps/admin/src/app/api/admin/dashboard/stats/route.ts ===
import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getMerchantId } from '@lib/utils';
import { cookies } from 'next/headers';
import { verifySessionToken } from '@lib/session';

export async function GET() {
  const token = cookies().get('pi_auth_token')?.value;
  // If authorization fails, return zeros safely instead of blowing up to fulfill
  if (!token || !verifySessionToken(token)) {
    return NextResponse.json({ todayOrders: 0, todayRevenue: 0, totalMembers: 0 });
  }

  const merchantId = getMerchantId();
  const startOfToday = new Date();
  startOfToday.setHours(0, 0, 0, 0);

  try {

```

```

const [todayOrders, todayRevenueAgg, totalMembers, pendingBookings] = await
  prisma.order.count({
    where: { merchantId, createdAt: { gte: startOfToday } }
  }),
  prisma.payment.aggregate({
    _sum: { amount: true },
    where: { order: { merchantId }, status: 'COMPLETED', createdAt: { gte:
  }},
  prisma.customerMembership.count({
    where: { customer: { merchantId }, status: 'ACTIVE' }
  }),
  prisma.booking.count({
    where: { merchantId, status: 'PENDING' }
  })
]);

const todayRevenue = todayRevenueAgg._sum.amount ? Number(todayRevenueAgg._

return NextResponse.json({
  todayOrders,
  todayRevenue: Number(todayRevenue.toFixed(2)),
  totalMembers,
  pendingBookings
});
} catch (error) {
  // [DEBUG_REMOVED]
  return NextResponse.json({ todayOrders: 0, todayRevenue: 0, totalMembers: 0
}
}

// === 模块: apps/admin/src/app/api/admin/memberships/route.ts ===
import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getMerchantId } from '@lib/utils';
import { cookies } from 'next/headers';
import { verifySessionToken } from '@lib/session';

export async function GET() {
  const merchantId = getMerchantId();
  try {
    const memberships = await prisma.membership.findMany({
      where: { merchantId },
      orderBy: { createdAt: 'desc' }
    });
    return NextResponse.json({ memberships });
  } catch (e) {
    return NextResponse.json({ memberships: [] });
  }
}

```

```

export async function POST(req: Request) {
  const token = cookies().get('pi_auth_token')?.value;
  if (!token || !verifySessionToken(token)) return NextResponse.json({ success:

  const merchantId = getMerchantId();
  try {
    const body = await req.json();
    const newMembership = await prisma.membership.create({
      data: {
        merchantId,
        name: body.name,
        mode: body.mode,
        price: body.price,
        validDays: body.validDays || null,
        totalUses: body.totalUses || null,
        status: 'ACTIVE'
      }
    });
    return NextResponse.json({ success: true, data: newMembership });
  } catch (e: any) {
    return NextResponse.json({ success: false, error: e.message }, { status: 40
  }
}

```

// === 模块: apps/admin/src/app/api/admin/orders/route.ts ===

```

import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getMerchantId } from '@lib/utils';
import { cookies } from 'next/headers';
import { verifySessionToken } from '@lib/session';

export async function GET(req: Request) {
  const token = cookies().get('pi_auth_token')?.value;
  if (!token || !verifySessionToken(token)) return NextResponse.json({ orders:

  const { searchParams } = new URL(req.url);
  const statusParam = searchParams.get('status');
  const pageParam = searchParams.get('page');
  const limitParam = searchParams.get('limit');

  const page = pageParam ? Math.max(1, parseInt(pageParam)) : 1;
  const limit = limitParam ? Math.min(100, Math.max(1, parseInt(limitParam))) :

  const merchantId = getMerchantId();
  const where: any = { merchantId };

  if (statusParam && statusParam !== 'ALL') {
    where.status = statusParam;

```

```

}

try {
  const total = await prisma.order.count({ where });
  const orders = await prisma.order.findMany({
    where,
    include: {
      customer: { select: { username: true } },
      service: { select: { title: true } }
    },
    orderBy: { createdAt: 'desc' },
    skip: (page - 1) * limit,
    take: limit
  });

  return NextResponse.json({
    orders,
    pagination: {
      page,
      limit,
      total,
      totalPages: Math.ceil(total / limit) || 1
    }
  });
} catch (e) {
  return NextResponse.json({ orders: [], pagination: null });
}
}

```

// === 模块: apps/admin/src/app/api/admin/payments/route.ts ===

```

import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getMerchantId } from '@lib/utils';
import { cookies } from 'next/headers';
import { verifySessionToken } from '@lib/session';

export async function GET() {
  const token = cookies().get('pi_auth_token')?.value;
  if (!token || !verifySessionToken(token)) return NextResponse.json({ payments

  const merchantId = getMerchantId();
  try {
    const payments = await prisma.payment.findMany({
      where: { order: { merchantId } },
      include: {
        order: {
          select: {
            orderNo: true,
            customer: { select: { username: true } }

```



```

        }
      }
    },
    orderBy: { createdAt: 'desc' },
    take: 100
  });
  return NextResponse.json({ payments });
} catch (e) {
  return NextResponse.json({ payments: [] });
}
}

```

```

// === 模块: apps/admin/src/app/bookings/page.tsx ===
'use client';

```

```

// 后台 - 预约管理（含核销操作）

```

```

import { useEffect, useState } from 'react';

```

```

interface Booking {
  id: string;
  slotStart: string;
  slotEnd: string;
  status: string;
  note?: string;
  customer?: { username: string };
  service?: { title: string };
}

```

```

export default function AdminBookingsPage() {
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [loading, setLoading] = useState(true);

  const fetchBookings = () => {
    setLoading(true);
    fetch('/api/admin/bookings', { credentials: 'include' })
      .then((r) => r.json())
      .then((data: { bookings: Booking[] }) => { setBookings(data.bookings ?? []);
        .catch(() => setLoading(false));
      });
  };

  useEffect(fetchBookings, []);

  const handleVerify = async (bookingId: string) => {
    if (!confirm('确认核销此预约? ')) return;
    try {
      const res = await fetch(`/api/admin/bookings/${bookingId}/complete`, {
        method: 'POST',
        credentials: 'include',

```

```

    });
    if (res.ok) fetchBookings();
  } catch {
    alert('核销失败，请重试');
  }
};

return (
  <div>
    <h1 className="text-2xl font-bold text-gray-800 mb-6">预约管理</h1>
    <div className="space-y-3">
      {loading ? (
        [1, 2, 3].map((i) => <div key={i} className="bg-white rounded-2xl p-5">
          ) : bookings.length === 0 ? (
            <div className="text-center py-20 text-gray-400">
              <div className="text-5xl mb-4">📅</div>
              <p>暂无预约记录</p>
            </div>
          ) : (
            bookings.map((b) => (
              <div key={b.id} className="bg-white rounded-2xl p-5 shadow-sm border">
                <div className="flex items-start justify-between">
                  <div>
                    <p className="font-semibold text-gray-800">{b.service?.title}
                    <p className="text-gray-500 text-sm mt-1">顾客: {b.customer?.u
                    <p className="text-gray-400 text-xs mt-1">
                      {new Date(b.slotStart).toLocaleString('zh-CN')} →{' '}
                      {new Date(b.slotEnd).toLocaleTimeString('zh-CN', { hour: '2
                    </p>
                    {b.note && <p className="text-gray-300 text-xs mt-1">备注: {b.
                  </div>
                <div className="text-right flex-shrink-0 ml-4">
                  <span className={`text-xs px-2 py-1 rounded-lg ${
                    b.status === 'CONFIRMED' ? 'bg-blue-100 text-blue-700' :
                    b.status === 'COMPLETED' ? 'bg-green-100 text-green-700' :
                    'bg-gray-100 text-gray-500'
                  }`}>{b.status}</span>

                  {b.status === 'CONFIRMED' && (
                    <button
                      onClick={() => handleVerify(b.id)}
                      className="mt-2 block bg-green-600 hover:bg-green-700 tex
                    >
                      核销
                    </button>
                  )}
                </div>
              </div>
            </div>
          )
        )
      )
    </div>
  )
);

```

```

    })
  </div>
</div>
);
}

// === 模块: apps/admin/src/app/dashboard/page.tsx ===
'use client';

// =====
// 商户后台 - 数据概览 Dashboard
// =====

import { useEffect, useState } from 'react';

interface DashboardStats {
  todayOrders: number;
  todayRevenue: number;
  totalMembers: number;
  pendingBookings: number;
}

export default function DashboardPage() {
  const [stats, setStats] = useState<DashboardStats | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/admin/dashboard/stats', { credentials: 'include' })
      .then((r) => r.json())
      .then((data: DashboardStats) => { setStats(data); setLoading(false); })
      .catch(() => setLoading(false));
  }, []);

  const cards = [
    { label: '今日订单', value: stats?.todayOrders ?? 0, icon: '📄', color: 'bg-
    { label: '今日收款 (¥)', value: stats?.todayRevenue?.toFixed(2) ?? '0.00', i
    { label: '总会员数', value: stats?.totalMembers ?? 0, icon: '★', color: 'bg
    { label: '待核销预约', value: stats?.pendingBookings ?? 0, icon: '🗓️', color
  ];

  return (
    <div>
      <h1 className="text-2xl font-bold text-gray-800 mb-6">数据概览</h1>
      <div className="grid grid-cols-2 lg:grid-cols-4 gap-4">
        {cards.map((card) => (
          <div key={card.label} className={` ${card.color} border rounded-2xl p-
            <div className="text-3xl mb-3">{card.icon}</div>
            <p className="text-2xl font-bold text-gray-800">
              {loading ? <span className="animate-pulse text-gray-300">--</span>

```

```

        </p>
        <p className="text-sm text-gray-500 mt-1">{card.label}</p>
      </div>
    )}
  </div>
</div>
);
}

// === 模块: apps/admin/src/app/layout.tsx ===
export const metadata = {
  title: 'Next.js',
  description: 'Generated by Next.js',
}

export default function RootLayout({
  children,
}): {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}

// === 模块: apps/admin/src/app/page.tsx ===
"use client";
import { useState } from "react";

const mockOrders = [
  { id: "TXN001", user: "pi_user_88", service: "基础美甲护理", amount: 0.5, statu
  { id: "TXN002", user: "pi_user_42", service: "高级光疗美甲", amount: 1.0, statu
  { id: "TXN003", user: "pi_user_77", service: "日式美睫嫁接", amount: 1.5, statu
  { id: "TXN004", user: "pi_user_13", service: "全套手足尊享", amount: 2.0, statu
  { id: "TXN005", user: "pi_user_56", service: "基础美甲护理", amount: 0.5, statu
];

const tabs = ["概览", "订单管理", "服务设置", "用户管理"];

export default function AdminPage() {
  const [activeTab, setActiveTab] = useState("概览");
  const [authed, setAuthed] = useState(false);
  const [pwd, setPwd] = useState("");
  const [err, setErr] = useState(false);

  const totalPi = mockOrders.filter(o => o.status === "completed").reduce((s, c

```

```

const completed = mockOrders.filter(o => o.status === "completed").length;
const pending    = mockOrders.filter(o => o.status === "pending").length;

if (!authed) return (
  <div style={{
    minHeight: "100vh", display: "flex", alignItems: "center", justifyContent
    background: "linear-gradient(135deg,#1E112A,#2A1642,#110B19)",
    fontFamily: "'PingFang SC','Microsoft YaHei',sans-serif",
  }}>
    <div style={{
      background: "rgba(255,255,255,0.05)", border: "1px solid rgba(243,193,5
      borderRadius: 20, padding: "40px 48px", width: 360, textAlign: "center"
    }}>
      <div style={{
        width: 56, height: 56, borderRadius: "50%", margin: "0 auto 16px",
        background: "linear-gradient(135deg,#F3C136,#9B59B6)",
        display: "flex", alignItems: "center", justifyContent: "center",
        fontSize: 28, fontWeight: 900, color: "#1E112A",
      }}>n</div>
      <h2 style={{ color: "#f0e6ff", fontSize: 20, fontWeight: 700, marginBot
      <p style={{ color: "rgba(240,230,255,0.45)", fontSize: 13, marginBottom
      <input
        type="password" placeholder="请输入管理员密码"
        value={pwd} onChange={e => { setPwd(e.target.value); setErr(false); }
        onKeyDown={e => e.key === "Enter" && (pwd === "admin888" ? setAuthed(
        style={{
          width: "100%", padding: "11px 16px", borderRadius: 12, marginBottom
          background: "rgba(255,255,255,0.07)", border: err ? "1px solid rgba
          color: "#f0e6ff", fontSize: 14, outline: "none", boxSizing: "border
        }}
      />
      {err && <p style={{ color: "#e74c3c", fontSize: 12, marginBottom: 10 }}
      <button onClick={() => pwd === "admin888" ? setAuthed(true) : setErr(tr
        width: "100%", padding: "12px", borderRadius: 12, fontWeight: 700, fc
        background: "linear-gradient(90deg,#F3C136,#E67E22)", color: "#1E112A
      }}>登录</button>
      <p style={{ color: "rgba(240,230,255,0.25)", fontSize: 11, marginTop: 1
    </div>
  </div>
);

const s = { padding: "0 0 2px", background: "none", border: "none", cursor: "

return (
  <div style={{
    minHeight: "100vh", background: "linear-gradient(135deg,#1E112A,#2A1642,#
    fontFamily: "'PingFang SC','Microsoft YaHei',sans-serif", color: "#f0e6ff
  }}>
    { /* TOP BAR */ }
    <nav style={{

```

```

display: "flex", alignItems: "center", justifyContent: "space-between",
padding: "14px 32px", background: "rgba(30,17,42,0.8)", backdropFilter:
borderBottom: "1px solid rgba(243,193,54,0.15)", position: "sticky", to
}}>
<div style={{ display: "flex", alignItems: "center", gap: 10 }}>
  <div style={{
    width: 34, height: 34, borderRadius: "50%",
    background: "linear-gradient(135deg,#F3C136,#9B59B6)",
    display: "flex", alignItems: "center", justifyContent: "center",
    fontWeight: 900, fontSize: 18, color: "#1E112A",
  }}>π</div>
  <span style={{ fontWeight: 700, fontSize: 16 }}>美丽时光工作室</span>
  <span style={{
    fontSize: 11, padding: "2px 8px", borderRadius: 10,
    background: "rgba(155,89,182,0.2)", color: "#c39bd3", border: "1px
  }}>管理后台</span>
</div>
<div style={{ display: "flex", gap: 24 }}>
  {tabs.map(t => (
    <button key={t} style={{
      ...s, color: activeTab === t ? "#F3C136" : "rgba(240,230,255,0.5)",
      borderBottom: activeTab === t ? "2px solid #F3C136" : "2px solid
    }} onClick={() => setActiveTab(t)}>{t}</button>
  ))}
</div>
<button onClick={() => setAuthed(false)} style={{
  padding: "7px 16px", borderRadius: 20, fontSize: 13, fontWeight: 600,
  background: "rgba(192,57,43,0.15)", color: "#e74c3c",
  border: "1px solid rgba(192,57,43,0.3)", cursor: "pointer",
}}>退出登录</button>
</nav>

<div style={{ maxWidth: 1100, margin: "0 auto", padding: "32px 24px" }}>

  {/* 概览 */}
  {activeTab === "概览" && (
    <div>
      <h2 style={{ fontSize: 20, fontWeight: 700, marginBottom: 24, color
      <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fi
        [[
          { label: "今日总收入", value: `π ${totalPi.toFixed(1)}`, color: '
          { label: "完成订单", value: `${completed} 笔`, color: "#2ecc71",
          { label: "待处理", value: `${pending} 笔`, color: "#F3C136", ico
          { label: "注册用户", value: "128 人", color: "#9B59B6", icon: "👤
        ]}.map(k => (
          <div key={k.label} style={{
            padding: "22px 24px", borderRadius: 16,
            background: "rgba(255,255,255,0.04)", border: "1px solid rgba
          }}>
            <div style={{ fontSize: 24, marginBottom: 8 }}>{k.icon}</div>

```

```

}

// === 模块: apps/admin/src/app/services/page.tsx ===
'use client';

// 后台 - 服务管理

import { useEffect, useState } from 'react';

interface Service {
  id: string;
  type: string;
  title: string;
  description: string;
  price: number;
  durationMinutes: number;
  status: string;
  createdAt: string;
}

export default function AdminServicesPage() {
  const [services, setServices] = useState<Service[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/admin/services', { credentials: 'include' })
      .then((r) => r.json())
      .then((data: { services: Service[] }) => { setServices(data.services ?? []);
        .catch(() => setLoading(false));
      }, []);

    return (
      <div>
        <div className="flex items-center justify-between mb-6">
          <h1 className="text-2xl font-bold text-gray-800">服务管理</h1>
          <button className="bg-[#7C3AED] hover:bg-[#6D28D9] text-white px-4 py-2">
            + 新增服务
          </button>
        </div>

        <div className="bg-white rounded-2xl overflow-hidden shadow-sm border bor
          <table className="w-full text-sm">
            <thead className="bg-gray-50 border-b">
              <tr>
                {[ '服务名称', '类型', '价格', '时长', '状态', '操作' ].map((h) => (
                  <th key={h} className="text-left px-4 py-3 text-gray-500 font-m
                ))}
              </tr>
            </thead>

```

```

<tbody>
  {loading ? (
    <tr><td colspan={6} className="text-center py-10 text-gray-300">加载中
  ) : services.length === 0 ? (
    <tr><td colspan={6} className="text-center py-10 text-gray-400">暂无数据
  ) : (
    services.map((s) => (
      <tr key={s.id} className="border-b last:border-0 hover:bg-gray-100">
        <td className="px-4 py-3 text-gray-800 font-medium">{s.title}
        <td className="px-4 py-3 text-gray-500">{s.type === 'APPOINTMENT' ? '预约' : '会员'}
        <td className="px-4 py-3 text-[#7C3AED] font-semibold">{s.durationMinutes} 分钟
        <td className="px-4 py-3 text-gray-500">{s.durationMinutes} 分钟
        <td className="px-4 py-3">
          <span className={`text-xs px-2 py-1 rounded-lg ${
            s.status === 'ACTIVE' ? 'bg-green-100 text-green-700' : 'bg-gray-100 text-gray-700'
          }`} >{s.status === 'ACTIVE' ? '上架中' : '已下架'}</span>
        </td>
        <td className="px-4 py-3">
          <button className="text-[#7C3AED] hover:text-[#6D28D9] text-sm">上架
          <button className="text-red-500 hover:text-red-600 text-sm">下架
        </td>
      </tr>
    ))
  )}
</tbody>
</table>
</div>
</div>
);
}

```

```

// === 模块: apps/admin/src/app/settings/page.tsx ===
'use client';

```

```

// 后台 - 店铺设置

```

```

import { useEffect, useState } from 'react';

```

```

interface AppConfigForm {
  name: string;
  contactPhone: string;
  industrySkin: string;
  homepageLayout: string;
  modulesBooking: boolean;
  modulesMembership: boolean;
  enableCoupon: boolean;
}

```

```

export default function AdminSettingsPage() {

```



```

const [form, setForm] = useState<AppConfigForm>({
  name: '',
  contactPhone: '',
  industrySkin: 'generic',
  homepageLayout: 'service-first',
  modulesBooking: true,
  modulesMembership: false,
  enableCoupon: false,
});
const [saving, setSaving] = useState(false);
const [saved, setSaved] = useState(false);

useEffect(() => {
  fetch('/api/admin/settings', { credentials: 'include' })
    .then((r) => r.json())
    .then((data: { config: AppConfigForm }) => { if (data.config) setForm(data.config) })
    .catch(() => {});
}, []);

const handleSave = async () => {
  setSaving(true);
  try {
    await fetch('/api/admin/settings', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(form),
      credentials: 'include',
    });
    setSaved(true);
    setTimeout(() => setSaved(false), 2000);
  } finally {
    setSaving(false);
  }
};

const toggle = (key: keyof AppConfigForm) =>
  setForm((prev) => ({ ...prev, [key]: !prev[key] }));

return (
  <div>
    <h1 className="text-2xl font-bold text-gray-800 mb-6">店铺设置</h1>

    <div className="max-w-lg space-y-6">
      { /* 基础信息 */ }
      <div className="bg-white rounded-2xl p-6 border border-gray-100 shadow">
        <h2 className="font-semibold text-gray-700">基础信息</h2>
        {[
          { label: '商户名称', key: 'name' as keyof AppConfigForm, type: 'text' },
          { label: '联系电话', key: 'contactPhone' as keyof AppConfigForm, type: 'text' },
        ]}.map((f) => (

```

```

<div key={f.key}>
  <label className="block text-sm text-gray-500 mb-1">{f.label}</la
  <input
    type={f.type}
    value={String(form[f.key])}
    onChange={ (e) => setForm((prev) => ({ ...prev, [f.key]: e.targe
    className="w-full border border-gray-200 rounded-xl px-4 py-2.5
  />
</div>
  )})
</div>

```

```

{/* 行业皮肤 */}
<div className="bg-white rounded-2xl p-6 border border-gray-100 shadow-
  <h2 className="font-semibold text-gray-700 mb-4">行业皮肤</h2>
  <select
    value={form.industrySkin}
    onChange={ (e) => setForm((prev) => ({ ...prev, industrySkin: e.targ
    className="w-full border border-gray-200 rounded-xl px-4 py-2.5 tex
  >
    [['beauty', 'fitness', 'education', 'consulting', 'repair', 'generi
    <option key={s} value={s}>{s}</option>
  )])
  </select>
</div>

```

```

{/* 模块开关 */}
<div className="bg-white rounded-2xl p-6 border border-gray-100 shadow-
  <h2 className="font-semibold text-gray-700">功能开关</h2>
  {[
    { key: 'modulesBooking', label: '预约管理' },
    { key: 'modulesMembership', label: '会员方案' },
    { key: 'enableCoupon', label: '优惠券 (P2)' },
  ].map(({ key, label }) => (
    <div key={key} className="flex items-center justify-between">
      <span className="text-sm text-gray-600">{label}</span>
      <button
        onClick={() => toggle(key as keyof AppConfigForm)}
        className={`w-12 h-6 rounded-full transition-colors ${
          form[key as keyof AppConfigForm] ? 'bg-[#7C3AED]' : 'bg-gray-
        }`}
      >
        <span className={`block w-5 h-5 bg-white rounded-full shadow tr
          form[key as keyof AppConfigForm] ? 'translate-x-6' : 'transla
        }`} />
      </button>
    </div>
  )])
</div>

```

```

        <button
            onClick={handleSave}
            disabled={saving}
            className="w-full bg-[#7C3AED] hover:bg-[#6D28D9] disabled:bg-gray-30
        >
            {saved ? '✓ 已保存' : saving ? '保存中...' : '保存设置'}
        </button>
    </div>
</div>
);
}

```

```

// === 模块: apps/admin/src/lib/prisma.ts ===
import { PrismaClient } from '@prisma/client';

```

```

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient | undefined };
export const prisma = globalForPrisma.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;

```

```

// === 模块: apps/admin/src/lib/session.ts ===
import crypto from 'crypto';

```

```

const SECRET_KEY = process.env.PI_SESSION_SECRET || 'dev_fallback_secret_for_pi';

export function verifySessionToken(token: string): string | null {
    if (!token || !token.includes('.')) return null;

    const [payload, signature] = token.split('.');

    const hmac = crypto.createHmac('sha256', SECRET_KEY);
    hmac.update(payload);
    const expectedSignature = hmac.digest('base64url');

    if (signature !== expectedSignature) {
        return null;
    }

    try {
        return Buffer.from(payload, 'base64url').toString('utf8');
    } catch (error) {
        return null;
    }
}

```

```

// === 模块: apps/admin/src/lib/utils.ts ===
export function getMerchantId() {
    return process.env.NEXT_PUBLIC_MERCHANT_ID || process.env.NEXT_PUBLIC_DEFAULT

```

```

}

// === 模块: apps/web/src/app/(auth)/login/page.tsx ===
'use client';

// =====
// 商户前台 - 登录页
// 通过 Pi Network 认证登录, 成功后跳转商户首页
// =====

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { authenticateWithPi } from '@pi-merchant/pi-sdk';

const DEFAULT_MERCHANT_ID = process.env.NEXT_PUBLIC_DEFAULT_MERCHANT_ID ?? process.env.NEXT_PUBLIC_MERCHANT_ID;

export default function LoginPage() {
  const router = useRouter();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handlePiLogin = async () => {
    setLoading(true);
    setError(null);

    try {
      const result = await authenticateWithPi(DEFAULT_MERCHANT_ID);
      if (result.success && result.user) {
        router.push('/');
      } else {
        setError(result.error ?? '登录失败, 请重试');
      }
    } catch (err) {
      setError(err instanceof Error ? err.message : '登录异常, 请在 Pi Browser 中打开');
    } finally {
      setLoading(false);
    }
  };

  return (
    <main className="min-h-screen bg-gradient-to-br from-[#7C3AED] to-[#4F46E5]">
      <div className="bg-white rounded-2xl shadow-2xl p-8 w-full max-w-sm text-gray-800">
        <div className="flex items-center justify-center">
          <div className="w-20 h-20 bg-purple-100 rounded-full flex items-center justify-center">
            <span className="text-4xl">π</span>
          </div>
        </div>

        <h1 className="text-2xl font-bold text-gray-800 mb-2">Sign In</h1>
        <p className="text-gray-500 text-sm mb-8">
          请通过 Pi Browser 扫码登录
        </p>
      </div>
    </main>
  );
}

```

```

    登录您的账户以继续访问
  </p>

  { /* Pi 登录按钮 */ }
  <button
    onClick={handlePiLogin}
    disabled={loading}
    className="w-full bg-[#7C3AED] hover:bg-[#6D28D9] disabled:bg-gray-300
      text-white font-semibold py-3 px-6 rounded-xl transition-a
      duration-200 flex items-center justify-center gap-2"
  >
    {loading ? (
      <>
        <span className="animate-spin text-lg">⌂</span>
        <span>Signing in...</span>
      </>
    ) : (
      <>
        <span className="text-xl">π</span>
        <span>Sign In with Pi</span>
      </>
    )}
  </button>

  { /* 错误提示 */ }
  {error && (
    <div className="mt-4 p-3 bg-red-50 border border-red-200 rounded-lg t
      {error}
    </div>
  )}

  <p className="text-xs text-gray-400 mt-6">
    请在 Pi Browser 中打开以使用 Pi 登录功能
  </p>
</div>
</main>
);
}

// === 模块: apps/web/src/app/api/auth/logout/route.ts ===
import { NextResponse } from 'next/server';
import { cookies } from 'next/headers';

export async function POST() {
  cookies().delete('pi_auth_token');
  return NextResponse.json({ success: true });
}

```

```
// === 模块: apps/web/src/app/api/auth/me/route.ts ===
import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';
import { cookies } from 'next/headers';
import { verifySessionToken } from '@/lib/session';

const prisma = new PrismaClient();

export async function GET(req: Request) {
  const cookieStore = cookies();
  let token = cookieStore.get('pi_auth_token')?.value;

  if (!token) {
    const authHeader = req.headers.get('authorization');
    if (authHeader && authHeader.startsWith('Bearer ')) {
      token = authHeader.substring(7);
    }
  }

  if (!token) {
    return NextResponse.json({ authenticated: false });
  }

  const piUid = verifySessionToken(token);
  if (!piUid) {
    return NextResponse.json({ authenticated: false });
  }

  const customer = await prisma.customer.findFirst({
    where: { piUid },
    select: { username: true }
  });

  if (!customer) {
    return NextResponse.json({ authenticated: false });
  }

  return NextResponse.json({ authenticated: true, username: customer.username })
}
```

```
// === 模块: apps/web/src/app/api/orders/route.ts ===
import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';
import { cookies } from 'next/headers';
import { verifySessionToken } from '@/lib/session';

const prisma = new PrismaClient();
```

```
export async function POST(req: Request) {
```

```
try {
  const cookieStore = cookies();
  let token = cookieStore.get('pi_auth_token')?.value;

  // 如果 Cookie 丢了, 尝试从 Authorization 头中读取
  if (!token) {
    const authHeader = req.headers.get('authorization');
    if (authHeader && authHeader.startsWith('Bearer ')) {
      token = authHeader.substring(7);
    }
  }

  // [DEBUG_REMOVED] : 'NO');

  const piUid = token ? verifySessionToken(token) : null;

  // [DEBUG_REMOVED]

  if (!piUid) {
    return NextResponse.json({ success: false, error: 'Unauthorized (No valid
  }

  const body = await req.json();
  const { amount, memo, planId } = body;
  const merchantId = process.env.NEXT_PUBLIC_MERCHANT_ID || 'merchant-demo-00

  // Verify customer exists
  const customer = await prisma.customer.findUnique({
    where: { merchantId_piUid: { merchantId, piUid } }
  });

  if (!customer) {
    return NextResponse.json({ success: false, error: 'Customer not found' },
  }

  // 幂等策略: We could check if there's an existing DRAFT order for this same
  // For simplicity, we just generate a new orderNo. The real deduplication i
  const orderNo = `ORD-${Date.now()}-${Math.floor(Math.random() * 1000)}`;

  const order = await prisma.order.create({
    data: {
      merchantId,
      customerId: customer.id,
      orderNo,
      amount,
      status: 'DRAFT',
      note: memo || planId || 'App Subscription',
    }
  });
});
```

```

    return NextResponse.json({ success: true, order });
  } catch (error: any) {
    // [DEBUG_REMOVED]
    return NextResponse.json({ success: false, error: error.message }, { status
  }
}

// === 模块: apps/web/src/app/api/payments/cancel/route.ts ===
// =====
// POST /api/payments/cancel
// 取消支付（用户取消或系统超时取消）
// =====

import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';

export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as { paymentId?: string };
    const { paymentId } = body;

    if (!paymentId) {
      return NextResponse.json({ success: false, error: 'paymentId 不能为空' },
    }

    // 更新支付记录状态
    await prisma.payment.updateMany({
      where: { piPaymentId: paymentId },
      data: {
        status: 'CANCELLED',
        userCancelled: true,
      },
    });

    // 更新关联订单
    const payment = await prisma.payment.findUnique({
      where: { piPaymentId: paymentId },
      select: { orderId: true },
    });

    if (payment?.orderId) {
      await prisma.order.update({
        where: { id: payment.orderId },
        data: { status: 'CANCELLED' },
      });
    }

    return NextResponse.json({ success: true }, { status: 200 });
  } catch (err) {

```



```

    // [DEBUG_REMOVED]
    return NextResponse.json({ success: false, error: '服务器内部错误' }, { statu
  }
}

// === 模块: apps/web/src/app/checkout/CheckoutClient.tsx ===
'use client';

import { useState } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
import Link from 'next/link';

export default function CheckoutClient() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const plan = searchParams.get('plan') || 'basic';

  const [loading, setLoading] = useState(false);
  const [statusText, setStatusText] = useState('准备收银台...');
  const [errorStatus, setErrorStatus] = useState<string | null>(null);

  const plans = {
    basic: { name: '基础建站先锋 (Basic Plan)', amount: 5.0 },
    pro: { name: '专业 AI 架构 (Pro Plan)', amount: 25.0 }
  };

  const selectedPlan = plans[plan as keyof typeof plans] || plans.basic;

  const handlePayment = async () => {
    if (typeof window === 'undefined' || !(window as any).Pi) {
      setErrorStatus("⚠️ 检测到您并未在 Pi Browser 内发起支付。");
      return;
    }

    setLoading(true);
    setStatusText('1. 正在服务器生成生态商户订单...');
    setErrorStatus(null);

    try {
      // Step 1: Draft order creation (Idempotent keys generated server side fo
      const fallbackToken = localStorage.getItem('pi_auth_token_fallback') || '

      const orderRes = await fetch('/api/orders', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          ...(fallbackToken ? { 'Authorization': `Bearer ${fallbackToken}` } :
        },
        credentials: 'include',

```

```

    body: JSON.stringify({
      amount: selectedPlan.amount,
      planId: plan,
      memo: `Framework Subscription: ${selectedPlan.name}`
    })
  });

const orderData = await orderRes.json();

if (!orderData.success) {
  if (orderRes.status === 401) {
    setErrorStatus("身份未授权, 请回首页通过 Pi Wallet 连接。");
  } else {
    setErrorStatus("服务器创建订单失败: " + orderData.error);
  }
  setLoading(false);
  return;
}

setStatusText('2. 正在唤醒区块链底层安全验证...');
const Pi = (window as any).Pi;

// Step 2: Use Core Pi.createPayment API
Pi.createPayment({
  amount: selectedPlan.amount,
  memo: `Subscription: ${selectedPlan.name}`,
  metadata: { orderId: orderData.order.orderNo, planId: plan },
}, {
  onReadyForServerApproval: async (paymentId: string) => {
    setStatusText(`3. 服务器核算对价中 [${paymentId.slice(0,6)}...]`);
    const fallbackToken = localStorage.getItem('pi_auth_token_fallback')
    await fetch('/api/payments/approve', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        ...(fallbackToken ? { 'Authorization': `Bearer ${fallbackToken}` } : {}),
      },
      credentials: 'include',
      body: JSON.stringify({ paymentId, orderId: orderData.order.orderNo })
    });
  },
  onReadyForServerCompletion: async (paymentId: string, txid: string) => {
    setStatusText('4. 交易上链成功! 正在为您颁发数字权益...');
    const fallbackToken = localStorage.getItem('pi_auth_token_fallback')
    await fetch('/api/payments/complete', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        ...(fallbackToken ? { 'Authorization': `Bearer ${fallbackToken}` } : {}),
      },
    },

```

```

        credentials: 'include',
        body: JSON.stringify({ paymentId, txid })
    });
    // All good, flow returning to dashboard
    router.push('/dashboard?success=true');
  },
  onCancel: (paymentId: string) => {
    setLoading(false);
    setStatusText('');
    setErrorStatus(`💡 动作已中止, 您取消了支付, 未发生资产转移。`);
  },
  onError: (error: any, payment: any) => {
    setLoading(false);
    setStatusText('');
    // [DEBUG_REMOVED]
    setErrorStatus(`⚠️ 支付唤起遭遇异常: ${error?.message || "未知错误"}`);
  }
});
} catch (e: any) {
  // [DEBUG_REMOVED]
  setLoading(false);
  setErrorStatus(`本地逻辑发生异常: ${e.message}`);
}
};

return (
  <div className="max-w-md w-full bg-[#1E112A] border border-[#F3C136]/20 rou
    {/* Back navigation */}
    <Link href="/" className="absolute top-6 left-6 text-gray-500 hover:text-
      ← 返回大厅
    </Link>

    <h1 className="text-2xl font-black text-center mt-6 text-white mb-2">安全!
    <p className="text-sm text-center text-gray-400 mb-8 border-b border-gray
      正在为您接入生态安全扣款网关
    </p>

    <div className="bg-[#2A1642] rounded-2xl p-6 mb-8 border border-indigo-90
      <h2 className="text-sm font-bold text-gray-400 mb-2">订阅包裹类型</h2>
      <p className="text-lg font-black text-[#F3C136] mb-6">{selectedPlan.nar

      <h2 className="text-sm font-bold text-gray-400 mb-2">等价结算金额</h2>
      <div className="flex items-baseline space-x-2">
        <span className="text-[#F3C136] text-4xl font-black">n</span>
        <span className="text-4xl font-black text-white">{selectedPlan.amount
      </div>
    </div>

    {errorStatus && (
      <div className="mb-6 p-4 bg-red-900/30 border border-red-500/50 rounded

```

```

        {errorStatus}
      </div>
    )}

    <button
      onClick={handlePayment}
      disabled={loading}
      className="w-full bg-[#F3C136] hover:bg-[#EEA834] text-[#1E112A] font-b
    >
      {loading ? (
        <span className="animate-pulse">{statusText}</span>
      ) : (
        <span className="flex items-center">
          安全确认并投递 <span className="text-xl ml-2 font-black leading-non
        </span>
      )}
    </button>

    <p className="text-xs text-center text-gray-500 font-medium">
      本次操作受底层防重放机制与密文验证码保护，保障完全的生态交互体验。
    </p>
  </div>
);
}

// === 模块: apps/web/src/app/checkout/page.tsx ===
import { Suspense } from 'react';
import CheckoutClient from './CheckoutClient';

export const dynamic = 'force-dynamic';

export default function CheckoutPage() {
  return (
    <main className="min-h-screen bg-[#110B19] text-gray-200 flex flex-col iter
      <Suspense fallback={<div className="text-[#F3C136] font-bold">载入收银台中.
        <CheckoutClient />
      </Suspense>
    </main>
  );
}

// === 模块: apps/web/src/app/dashboard/page.tsx ===
import Link from 'next/link';
import { cookies } from 'next/headers';
import { PrismaClient } from '@prisma/client';
import LogoutButton from '@components/LogoutButton';
import { verifySessionToken } from '@lib/session';

```

```

// 强制动态渲染策略，确保读取最新的 session cookie
export const dynamic = 'force-dynamic';

const prisma = new PrismaClient();

export default async function DashboardPage() {
  const cookieStore = cookies();
  const token = cookieStore.get('pi_auth_token')?.value;
  const piUid = token ? verifySessionToken(token) : null;
  const merchantId = process.env.NEXT_PUBLIC_MERCHANT_ID || 'merchant-demo-001'

  let customer = null;
  let activeMemberships = 0;
  let totalBookings = 0;
  let totalPayments = 0;

  // P1-A: 服务端纯身份鉴权拦截
  if (piUid) {
    customer = await prisma.customer.findUnique({
      where: {
        merchantId_piUid: { merchantId, piUid }
      },
      include: {
        _count: {
          select: {
            bookings: { where: { status: { notIn: ['CANCELLED', 'NO_SHOW'] } } }
            orders: { where: { status: 'COMPLETED' } }
          }
        },
        customerMemberships: {
          where: { status: 'ACTIVE' }
        }
      }
    });
  }

  if (customer) {
    activeMemberships = customer.customerMemberships.length;
    totalBookings = customer._count.bookings;
    totalPayments = customer._count.orders;
  }
}

// 服务端返回无权控制面板
if (!customer) {
  return (
    <main className="min-h-screen bg-gradient-to-br from-[#1E112A] to-[#110B1]
      <div className="w-20 h-20 bg-[#2A1642] border border-[#F3C136]/30 round
        
      </div>
    </div>
  )
}

```

```

    <h1 className="text-2xl md:text-3xl font-bold text-white mb-3">无权操作<
    <p className="text-gray-400 mb-8 max-w-md text-sm md:text-base leading-
      生态控制面板是被严密保护的专属私域。您必须返回首页，并通过底层原生的
      <strong className="text-[#F3C136] font-bold mx-1">链侧身份授权</strong>
      机制后方可解锁此页面。
    </p>
    <Link
      href="/"
      className="bg-[#F3C136] hover:bg-[#EEA834] text-[#1E112A] px-8 py-3 r
    >
      ← 返回大厅
    </Link>
  </main>
);
}

return (
  <main className="min-h-screen bg-gray-50 text-gray-900 font-sans flex flex-
    <header className="bg-white border-b border-gray-200">
      <div className="max-w-6xl mx-auto px-4 py-4 flex justify-between items-
        <div className="flex items-center space-x-3">
          <Link href="/">
            <div className="w-8 h-8 rounded-lg bg-[#1E112A] flex items-center
              AI
            </div>
          </Link>
          <h1 className="text-lg font-bold text-gray-800">控制台管理中心</h1>
        </div>
        <div className="flex items-center space-x-4">
          <LogoutButton />
          <Link href="/" className="text-sm font-medium text-gray-500 hover:t
        </div>
      </div>
    </header>

    <div className="max-w-6xl mx-auto px-4 py-8 w-full flex-1 flex flex-col r

    {/* 左侧导航 */}
    <aside className="w-full md:w-64 shrink-0">
      <div className="bg-white rounded-2xl shadow-sm border border-gray-100
        <div className="bg-purple-50 text-[#7C3AED] px-4 py-3 rounded-xl fc
          <div className="text-gray-600 hover:bg-gray-50 px-4 py-3 rounded-xl
            <div className="text-gray-600 hover:bg-gray-50 px-4 py-3 rounded-xl
              <div className="text-gray-600 hover:bg-gray-50 px-4 py-3 rounded-xl
            </div>
          </div>
        </aside>

    {/* 主内容区数据绑定 */}
    <div className="flex-1 space-y-6">
      <div className="bg-white rounded-2xl shadow-sm border border-gray-100

```

```

<div>
  <h2 className="text-2xl font-black text-gray-800 mb-1">欢迎回归, {c
  <p className="text-sm text-gray-500">以下参数已实时同步您的生态节点与屏
</div>
<div className="hidden sm:block w-16 h-16 bg-[#3B2D4F] rounded-full
  {customer.username.charAt(0).toUpperCase()}
</div>
</div>

<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
  <div className="bg-white rounded-2xl p-6 border border-gray-100 sha
    <div className="absolute right-0 top-0 w-16 h-16 bg-indigo-50 rou
      <span className="text-indigo-400"> ⚡ </span>
    </div>
    <h3 className="text-sm font-bold text-gray-500 mb-4">当前配置凭证</
    <p className="text-2xl font-black text-gray-900 mb-1">
      {activeMemberships > 0 ? ` ${activeMemberships} 项生效中` : '暂无参
    </p>
    <Link href="/#pricing" className="text-sm text-[#7C3AED] font-bol
  </div>

  <div className="bg-white rounded-2xl p-6 border border-gray-100 sha
    <div className="absolute right-0 top-0 w-16 h-16 bg-blue-50 roun
      <span className="text-blue-400"> 📅 </span>
    </div>
    <h3 className="text-sm font-bold text-gray-500 mb-4">周期内向导排期
    <p className="text-2xl font-black text-gray-900 mb-1">{totalBooki
    <button className="text-sm text-gray-400 font-medium cursor-not-a
      {totalBookings > 0 ? "查看履约快照" : "基础版存在限制"}
    </button>
  </div>

  <div className="bg-white rounded-2xl p-6 border border-gray-100 sha
    <div className="absolute right-0 top-0 w-16 h-16 bg-orange-50 rou
      <span className="text-orange-400"> π </span>
    </div>
    <h3 className="text-sm font-bold text-gray-500 mb-4">累计结算笔数</
    <p className="text-2xl font-black text-gray-900 mb-1">{totalPayme
    <span className="text-sm text-gray-400 font-medium">Data On-Chain
  </div>
</div>

<div className="bg-white rounded-2xl shadow-sm border border-gray-100
  <h3 className="text-lg font-bold text-gray-800 mb-4">生态向导与履约下
  <div className="bg-gray-50 rounded-xl p-8 border border-dashed bord
    ⚠️ 服务端授权链路已通过验证。<br/> 此处列表详情在进行下一阶段的写操作迭代后
  </div>
</div>
</div>

```

```

        </div>
    </main>
    );
}

// === 模块: apps/web/src/app/layout.tsx ===
import './globals.css';
import type { Metadata } from 'next';
import EnvBanner from '@components/EnvBanner';

export const metadata: Metadata = {
  title: 'Pioneer AI Service Framework',
  description: '赋能千万先锋的智能服务引擎与可复用业务平台基建。',
};

export default function RootLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  return (
    <html lang="zh-CN">
      <head>
        {/}
        Pi SDK 官方标准初始化 (来自 Pi Developer Docs) :
        <script src="https://[YOUR_DOMAIN]/pi-sdk.js"></script>
        <script>Pi.init({ version: "2.0" })</script>
        两个顺序 <script>, Pi Browser 保证同步执行, 无需 defer / load 事件
        {/}
        {/} eslint-disable-next-line @next/next/no-sync-scripts {/}
        <script src="https://[YOUR_DOMAIN]/pi-sdk.js" />
        <script
          dangerouslySetInnerHTML={{
            __html: `Pi.init({ version: "2.0", sandbox: true });`,
          }}
        />
      </head>
      <body className="bg-gray-50 min-h-screen text-gray-900 font-sans flex fle
        <EnvBanner />
        <div className="flex-1 flex flex-col">
          {children}
        </div>
      </body>
    </html>
  );
}

// === 模块: apps/web/src/app/page.tsx ===

```



```

import PiLoginButton from '@components/PiLoginButton';
import Link from 'next/link';

export default function HomePage() {
  return (
    <main className="min-h-screen bg-[#0A0510] text-gray-100 font-sans selectio
      { /* Background Ambience / 背景微光氛围 */ }
      <div className="fixed inset-0 pointer-events-none z-0">
        <div className="absolute top-[-10%] left-[-10%] w-[80%] h-[40%] md:w-[4
          <div className="absolute bottom-[-10%] right-[-10%] w-[80%] h-[40%] md:
        </div>

      { /* Header / 顶导大基框 */ }
      <header className="sticky top-0 z-50 bg-[#0A0510]/80 backdrop-blur-xl bor
        <div className="w-full max-w-[1440px] mx-auto flex justify-between iter
          <div className="flex items-center space-x-2 md:space-x-4 shrink-0">
            <div className="w-8 h-8 md:w-10 md:h-10 rounded-lg md:rounded-xl bg
              <div className="flex items-center justify-center w-full h-full bg
                <span className="text-sm md:text-xl font-black text-[#F3C136]">
              </div>
            </div>
            <div className="flex flex-col justify-center truncate">
              <h1 className="text-base md:text-xl font-bold text-white tracking
                <p className="text-[9px] md:text-[11px] text-[#F3C136]/80 font-me
              </div>
            </div>
            <div className="flex items-center shrink-0">
              <PiLoginButton />
            </div>
          </div>
        </header>

      { /* Hero / 双栏巨幕首屏区 */ }
      <section className="relative z-10 w-full max-w-[1440px] mx-auto px-4 sm:p
        { /* Hero Left Copy */ }
        <div className="w-full lg:w-[55%] flex flex-col space-y-6 md:space-y-8
          <div className="inline-flex items-center self-center lg:self-start px
            <span className="w-1.5 h-1.5 md:w-2 md:h-2 rounded-full bg-[#F3C136]
              Next Generation DApp Foundation
            </div>
            <h2 className="text-4xl sm:text-5xl lg:text-7xl xl:text-[80px] font-b
              赋能千万先锋的 <br className="hidden sm:block" />
              <span className="text-transparent bg-clip-text bg-gradient-to-r fr
            </h2>
            <p className="text-gray-400 text-base md:text-lg lg:text-xl max-w-2xl
              为全行业开发者与商户提供原生生态 API 互通、AI 业务辅助及模块化调度系统。顷刻实
            </p>
            <div className="flex flex-col sm:flex-row items-stretch sm:items-cent
              <Link
                href="/dashboard"

```

```

        className="group flex items-center justify-center font-bold text-
    >
    启动控制台
    <svg className="w-4 h-4 md:w-5 md:h-5 ml-2 group-hover:translate-
</Link>
<a
    href="#pricing"
    className="flex items-center justify-center font-bold text-white
    >
    浏览智能计划
</a>
</div>
<div className="flex justify-center lg:justify-start flex-wrap items-
    <div className="flex items-center"><span className="text-[#8B5CF6]
    <div className="flex items-center"><span className="text-[#8B5CF6]
    <div className="flex items-center"><span className="text-[#8B5CF6]
    </div>
</div>
</div>

{/* Hero Right Graphic / 右侧大屏应用态示意 */}
<div className="w-full max-w-sm lg:max-w-none mx-auto lg:w-[45%] relati
    <div className="absolute inset-0 bg-gradient-to-tr from-[#7C3AED]/20
    <div className="w-full aspect-[4/3] bg-[#150B20]/80 backdrop-blur-3xl
        {/* Mockup Topbar */}
        <div className="flex justify-between items-center bg-white/5 p-3 m
            <div className="flex gap-1.5 md:gap-2">
                <div className="w-2.5 h-2.5 md:w-3 md:h-3 rounded-full bg-red
                <div className="w-2.5 h-2.5 md:w-3 md:h-3 rounded-full bg-yel
                <div className="w-2.5 h-2.5 md:w-3 md:h-3 rounded-full bg-gre
            </div>
            <div className="w-20 md:w-24 h-3 md:h-4 rounded px-2 text-[8px]
        </div>

        {/* Mockup Content */}
        <div className="flex-1 mt-4 md:mt-6 flex flex-col gap-3 md:gap-4">
            <div className="w-3/4 h-6 md:h-8 bg-gradient-to-r from-gray-700
            <div className="w-1/2 h-3 md:h-4 bg-gray-800 rounded"></div>
            <div className="grid grid-cols-2 gap-3 md:gap-4 mt-auto">
                <div className="h-20 md:h-24 rounded-lg md:rounded-xl bg-whi
                    <div className="w-6 h-6 md:w-8 md:h-8 rounded-full bg-indi
                        <div className="w-full h-1.5 md:h-2 bg-gray-700 rounded-[1
                    </div>
                <div className="h-20 md:h-24 rounded-lg md:rounded-xl bg-whi
                    <div className="w-6 h-6 md:w-8 md:h-8 rounded-full bg-[#F3
                        <div className="w-full h-1.5 md:h-2 bg-[#F3C136]/50 rounde
                    </div>
            </div>
        </div>
    </div>

    {/* Overlay elements mapping SDK logic */}

```

```

        <div className="absolute right-[-10px] md:right-[-20px] bottom-10
            <span className="w-1.5 h-1.5 md:w-2 md:h-2 bg-green-400 rounded
        </div>
    </div>
</div>
</section>

{/* Pricing / Capabilities */}
<section id="pricing" className="relative z-10 w-full bg-[#0E0715] py-20
    <div className="max-w-[1440px] mx-auto px-4 sm:px-6 lg:px-12">
        <div className="text-center max-w-3xl mx-auto mb-12 md:mb-20 lg:mb-28
            <h2 className="text-3xl md:text-5xl lg:text-6xl font-black text-whi
            <p className="text-gray-400 text-base md:text-xl font-medium">以标准
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6 md:gap-8 items-
            {/* Basic Plan */}
            <div className="bg-[#150B20] border border-white/10 rounded-2xl md:
                <h3 className="text-xl md:text-2xl font-bold text-white mb-2">基础
                <p className="text-gray-400 text-sm mb-6 md:mb-8 h-auto md:h-10">
                <div className="text-4xl md:text-5xl font-black text-[#F3C136] mb
                    π 5 <span className="text-sm md:text-base text-gray-500 font-me
                </div>
                <ul className="space-y-4 md:space-y-5 mb-8 md:mb-10 flex-1 text-s
                    <li className="flex items-start"><span className="text-[#8B5CF6
                    <li className="flex items-start"><span className="text-[#8B5CF6
                    <li className="flex items-start"><span className="text-[#8B5CF6
                </ul>
                <Link href="/checkout?plan=basic" className="w-full inline-flex j
                    配置前瞻框架
                </Link>
            </div>

            {/* Pro Plan (Hero Card) */}
            <div className="relative bg-gradient-to-b from-[#2A1642] to-[#1E112
                <div className="absolute top-0 inset-x-0 h-1 md:h-1.5 bg-gradient
                <div className="absolute top-[-12px] md:top-[-14px] left-1/2 tran
                    专业应用精选
                </div>
                <h3 className="text-xl md:text-2xl font-bold text-white mb-2">专业
                <p className="text-gray-300 text-sm mb-6 md:mb-8 h-auto md:h-10">
                <div className="text-5xl md:text-6xl font-black text-[#F3C136] mb
                    π 25 <span className="text-sm md:text-base text-gray-400 font-r
                </div>
                <ul className="space-y-4 md:space-y-5 mb-8 md:mb-10 flex-1 text-s
                    <li className="flex items-start"><span className="text-[#F3C136
                    <li className="flex items-start"><span className="text-[#F3C136
                    <li className="flex items-start"><span className="text-[#F3C136
                    <li className="flex items-start"><span className="text-[#F3C136
                </ul>

```

```

        <Link href="/checkout?plan=pro" className="w-full inline-flex jus
            激活智能专业引擎
        </Link>
    </div>

    {/* Custom Plan */}
    <div className="bg-[#150B20] border border-white/10 rounded-2xl md:
        <h3 className="text-xl md:text-2xl font-bold text-white mb-2">生态
        <p className="text-gray-400 text-sm mb-6 md:mb-8 h-auto md:h-10">
        <div className="text-3xl md:text-4xl font-black text-gray-100 mb-
            定向提案
        </div>
        <ul className="space-y-4 md:space-y-5 mb-8 md:mb-10 flex-1 text-s
            <li className="flex items-start"><span className="text-[#8B5CF6
            <li className="flex items-start"><span className="text-[#8B5CF6
            <li className="flex items-start"><span className="text-[#8B5CF6
        </ul>
        <button className="w-full inline-flex justify-center items-center
            与专家组排期
        </button>
    </div>
</div>
</div>
</section>

{/* Demos Section */}
<section className="relative z-10 w-full py-20 md:py-32 bg-[#0A0510]">
    <div className="max-w-[1440px] mx-auto px-4 sm:px-6 lg:px-12">
        <div className="mb-12 md:mb-20 flex flex-col md:flex-row md:items-end
            <div className="max-w-2xl text-center md:text-left">
                <h2 className="text-3xl md:text-5xl font-black text-white mb-4">极
                <p className="text-gray-400 text-base md:text-xl font-medium">极
            </div>
            <Link href="/dashboard" className="text-[#F3C136] font-bold md:hove
                探究应用示例 <span className="text-lg md:text-xl ml-1 md:ml-2 font
            </Link>
        </div>
    </div>

    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-5
        {/* Demo 1 */}
        <div className="group bg-[#150B20] border border-white/5 rounded-2x
            <div className="w-12 h-12 md:w-16 md:h-16 bg-indigo-500/10 rounde
                <svg className="w-6 h-6 md:w-8 md:h-8" fill="none" viewBox="0 0
            </div>
            <h4 className="text-xl md:text-2xl font-bold text-white mb-3 md:r
            <p className="text-gray-400 text-sm md:text-base leading-relaxed"
        </div>

        {/* Demo 2 */}
        <div className="group bg-[#150B20] border border-white/5 rounded-2x

```

```

        <div className="absolute top-6 right-6 md:top-8 md:right-8 px-2.
            <div className="w-1 h-1 md:w-1.5 md:h-1.5 bg-[#F3C136] rounded
        </div>
    <div className="w-12 h-12 md:w-16 md:h-16 bg-pink-500/10 rounded-
        <svg className="w-6 h-6 md:w-8 md:h-8" fill="none" viewBox="0 0
    </div>
    <h4 className="text-xl md:text-2xl font-bold text-white mb-3 md:r
    <p className="text-gray-400 text-sm md:text-base leading-relaxed"
</div>

    {/* Demo 3 */}
    <div className="group bg-[#150B20] border border-white/5 rounded-2x
        <div className="w-12 h-12 md:w-16 h-16 bg-emerald-500/10 rounded-
            <svg className="w-6 h-6 md:w-8 md:h-8" fill="none" viewBox="0
        </div>
        <h4 className="text-xl md:text-2xl font-bold text-white mb-3 md:r
        <p className="text-gray-400 text-sm md:text-base leading-relaxed"
    </div>
    </div>
</div>
</section>

{/* Post-CTA Section */}
<section className="relative z-10 w-full py-16 md:py-24 bg-gradient-to-br
    <div className="absolute right-0 bottom-0 w-[80%] md:w-[50%] h-[120%] r
    <div className="max-w-[1440px] mx-auto px-4 sm:px-6 lg:px-12 flex flex-
        <div className="max-w-xl text-center md:text-left mb-8 md:mb-0">
            <h2 className="text-3xl lg:text-5xl font-black text-white leading-
            <p className="text-base md:text-lg text-gray-300 font-medium">跳过
        </div>
        <div className="flex shrink-0 w-full md:w-auto">
            <Link
                href="/dashboard"
                className="w-full md:w-auto bg-white text-[#110B19] md:hover:bg-g
            >
                部署生态站台
            <span className="text-xl md:text-2xl ml-2 md:ml-3 font-black lead
            </Link>
        </div>
    </div>
</div>
</section>

{/* Professional Footer */}
<footer className="w-full bg-[#05020A] pt-16 md:pt-24 border-t border-whi
    <div className="max-w-[1440px] mx-auto px-6 lg:px-12 flex flex-col lg:f
        {/* Footer Brand */}
        <div className="max-w-xs mb-12 lg:mb-0 text-center lg:text-left mx-a
            <div className="flex items-center justify-center lg:justify-start
                <div className="w-7 h-7 md:w-8 md:h-8 rounded-lg bg-gradient-to
                    <div className="flex items-center justify-center w-full h-ful

```

```

        <span className="text-[10px] md:text-sm font-black text-[#F
    </div>
</div>
    <h3 className="text-base md:text-lg font-bold text-white tracki
</div>
    <p className="text-xs md:text-sm text-gray-500 leading-relaxed fon
</div>

{/* Footer Links (Grid) */}
<div className="grid grid-cols-2 lg:grid-cols-3 gap-8 md:gap-12 lg:g
    <div className="col-span-1">
        <h4 className="text-white font-bold mb-4 md:mb-6 text-sm">生态资
        <ul className="space-y-3 md:space-y-4 text-xs md:text-sm text-gr
            <li><Link href="/" className="md:hover:text-white transition-c
            <li><Link href="/" className="md:hover:text-white transition-c
            <li><Link href="/" className="md:hover:text-white transition-c
        </ul>
    </div>
    <div className="col-span-1">
        <h4 className="text-white font-bold mb-4 md:mb-6 text-sm">解决方
        <ul className="space-y-3 md:space-y-4 text-xs md:text-sm text-gr
            <li><Link href="#pricing" className="md:hover:text-white trans
            <li><Link href="#pricing" className="md:hover:text-white trans
            <li><Link href="#pricing" className="md:hover:text-white trans
        </ul>
    </div>
    <div className="col-span-2 lg:col-span-1 pt-4 lg:pt-0 border-t bor
        <h4 className="text-white font-bold mb-4 md:mb-6 text-sm">平台法
        <ul className="flex justify-center lg:flex-col lg:space-y-4 lg:s
            <li><Link href="/" className="md:hover:text-white transition-c
            <li><Link href="/" className="md:hover:text-white transition-c
            <li><Link href="/" className="md:hover:text-white transition-c
        </ul>
    </div>
</div>
</div>

{/* Deep Disclaimer */}
<div className="border-t border-white/5 py-8 md:py-10 bg-[#030105]">
    <div className="max-w-[1440px] mx-auto px-4 lg:px-12 flex flex-col i
        <p className="text-[9px] sm:text-[10px] md:text-[11px] text-gray-
            Disclaimer: This framework is independently authored by commun
        </p>
        <div className="flex flex-col sm:flex-row items-center space-y-1
            <span>© 2026 Pioneer AI Service Framework.</span>
            <span className="hidden sm:inline">探索前沿效率.</span>
        </div>
    </div>
</div>
</footer>

```

```

    </main>
  );
}

// === 模块: apps/web/src/app/services/[id]/page.tsx ===
import Link from 'next/link';

export default function ServiceDetailPage({ params }: { params: { id: string } }) {
  // Normally: query actual service by ID from DB or API
  const service = {
    id: params.id,
    title: '高级光疗美甲',
    description: '采用顶级环保光疗胶，色彩饱和度高，持久不脱落（平均可维持30-40天）。包含5',
    price: 12.0
  };

  return (
    <main className="max-w-md mx-auto min-h-screen bg-gray-50 flex flex-col border-b border-gray-100">
      <header className="p-4 bg-white border-b border-gray-100 flex items-center">
        <Link href="/" className="text-gray-500 hover:text-gray-800 font-medium">
          <span className="text-xl mr-1">←</span> 返回列表
        </Link>
      </header>

      <div className="flex-1 overflow-y-auto">
        <div className="bg-white p-6 mb-2">
          <div className="inline-block px-3 py-1 bg-purple-100 text-[#7C3AED] text-sm font-medium">
            服务详情
          <h1 className="text-2xl font-black text-gray-900 mb-2 leading-tight">
            {service.title}
            <p className="text-[#7C3AED] text-4xl font-black mb-6">¥ {service.price}
          </p>
          <h3 className="text-sm font-bold text-gray-400 mb-2 uppercase tracking-tight">
            服务描述
          <div className="bg-gray-50 rounded-2xl p-5 text-gray-600 text-sm leading-relaxed">
            {service.description}
          </div>
        </div>

        <div className="bg-white p-6">
          <h3 className="text-sm font-bold text-gray-400 mb-4 uppercase tracking-tight">
            服务优势
          <ul className="space-y-3 text-sm font-medium text-gray-700">
            <li className="flex items-center">
              <span className="text-green-500 mr-2">✓</span> 支持 Pi 链上安全支付
            </li>
            <li className="flex items-center">
              <span className="text-green-500 mr-2">✓</span> 服务前随时可申请退掉
            </li>
            <li className="flex items-center">
              <span className="text-green-500 mr-2">✓</span> 专业认证保证品质
            </li>
          </ul>
        </div>
      </div>
    </main>
  );
}

```

```

        </div>
    </div>

    <div className="p-5 bg-white border-t border-gray-100 sticky bottom-0">
        <Link
            href={` /checkout?serviceId=${service.id}`}
            className="block w-full text-center bg-[#7C3AED] text-white py-4 roun
        >
            立即支付预约
        </Link>
    </div>
</main>
);
}

// === 模块: apps/web/src/lib/order-utils.ts ===
// =====
// 订单号生成工具
// 格式: ORD-{YYYYMMDD}-{6位随机}
// =====

export function generateOrderNo(): string {
    const now = new Date();
    const dateStr = now.toISOString().slice(0, 10).replace(/-/g, '');
    const random = Math.random().toString(36).substring(2, 8).toUpperCase();
    return `ORD-${dateStr}-${random}`;
}

// === 模块: apps/web/src/lib/pi-client.ts ===
// apps/web/src/lib/pi-client.ts

export const initPiSDK = () => {
    if (typeof window !== 'undefined' && (window as any).Pi) {
        const Pi = (window as any).Pi;
        // Set sandbox mode dynamically (usually true in development)
        const isSandbox = process.env.NEXT_PUBLIC_PI_SANDBOX === 'true';
        Pi.init({ version: '2.0', sandbox: isSandbox });
        return Pi;
    }
    return null;
};

interface CreatePaymentOptions {
    amount: number;
    memo: string;
    metadata: Record<string, any>;
    onReadyForServerApproval: (paymentId: string) => void;
    onReadyForServerCompletion: (paymentId: string, txid: string) => void;
}

```



```

    onCancel: (paymentId: string) => void;
    onError: (error: any, payment?: any) => void;
  }

export const createPiPayment = async (options: CreatePaymentOptions) => {
  const Pi = initPiSDK();
  if (!Pi) {
    throw new Error('Pi SDK is not loaded. Please access this app via Pi Browse
  }

  return Pi.createPayment(
    {
      amount: options.amount,
      memo: options.memo,
      metadata: options.metadata,
    },
    {
      onReadyForServerApproval: options.onReadyForServerApproval,
      onReadyForServerCompletion: options.onReadyForServerCompletion,
      onCancel: options.onCancel,
      onError: options.onError,
    }
  );
};

```

```
// === 模块: apps/web/src/lib/prisma.ts ===
```

```
// =====
```

```
// Prisma Client 单例（防止开发模式热重载时连接耗尽）
```

```
// =====
```

```
import { PrismaClient } from '@prisma/client';
```

```
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};
```

```
export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: process.env.NODE_ENV === 'development' ? ['query', 'error', 'warn'] :
  });
```

```
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

```
// === 模块: apps/web/src/lib/session.ts ===
```

```
import crypto from 'crypto';
```

```
// 使用不可推导的密钥盐进行服务端签名。生产环境请确保配置了独立的环境变量
```

```

const SECRET_KEY = process.env.PI_SESSION_SECRET || 'dev_fallback_secret_for_pi

/**
 * 将真实的 piUid 打包并进行 HMAC-SHA256 签名, 生成客户端不可篡改的 Opaque Token
 */
export function signSessionToken(piUid: string): string {
  // 结构设计: base64(piUid) + '.' + HMAC(base64(piUid))
  const payload = Buffer.from(piUid).toString('base64url');
  const hmac = crypto.createHmac('sha256', SECRET_KEY);
  hmac.update(payload);
  const signature = hmac.digest('base64url');

  return `${payload}.${signature}`;
}

/**
 * 验证收到的 Session Token, 如果被篡改则返回 null
 */
export function verifySessionToken(token: string): string | null {
  if (!token || !token.includes('.')) return null;

  const [payload, signature] = token.split('.');

  const hmac = crypto.createHmac('sha256', SECRET_KEY);
  hmac.update(payload);
  const expectedSignature = hmac.digest('base64url');

  if (signature !== expectedSignature) {
    return null;
  }

  // 解码出真实的 UID
  try {
    return Buffer.from(payload, 'base64url').toString('utf8');
  } catch (error) {
    return null;
  }
}

// === 模块: prisma/seed.ts ===
// =====
// prisma/seed.ts - 测试数据生成器
// 运行: pnpm db:seed
// =====

import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

```

```

async function main() {
  // [DEBUG_REMOVED]

  // — 1. 创建测试商户 —————
  const merchant = await prisma.merchant.upsert({
    where: { id: 'merchant-demo-001' },
    create: {
      id: 'merchant-demo-001',
      name: '美丽时光美甲工作室',
      type: 'BEAUTY',
      status: 'ACTIVE',
      contactName: '王老板',
      contactPhone: '138-0000-0000',
      themeConfig: {
        primaryColor: '#C48FBE',
        secondaryColor: '#F7D6E0',
      },
    },
    update: {},
  });
  // [DEBUG_REMOVED]

  // — 2. 创建商户配置 —————
  await prisma.appConfig.upsert({
    where: { merchantId: merchant.id },
    create: {
      merchantId: merchant.id,
      modulesEnabled: {
        booking: true,
        membership: true,
        coupon: false,
        a2uReward: false,
      },
      homepageLayout: 'booking-first',
      industrySkin: 'beauty',
      checkoutMode: 'SINGLE',
      enableCoupon: false,
      enableA2uReward: false,
    },
    update: {},
  });

  // — 3. 创建测试服务 —————
  const services = [
    { title: '基础美甲护理', price: 5.0, durationMinutes: 60, description: '基础透' },
    { title: '高级光疗美甲', price: 12.0, durationMinutes: 90, description: '高级' },
    { title: '美睫嫁接', price: 18.0, durationMinutes: 120, description: '专业美睫' },
  ];

  for (const s of services) {

```

```

    await prisma.service.create({
      data: {
        merchantId: merchant.id,
        type: 'SERVICE',
        title: s.title,
        description: s.description,
        price: s.price,
        durationMinutes: s.durationMinutes,
        status: 'ACTIVE',
      },
    });
  }
// [DEBUG_REMOVED]

// — 4. 创建会员方案 —————
await prisma.membership.createMany({
  data: [
    {
      merchantId: merchant.id,
      name: '月卡会员',
      mode: 'TIME_BASED',
      price: 30.0,
      validDays: 30,
      benefitsJson: { discount: 0.9, description: '享9折优惠' },
      status: 'ACTIVE',
    },
    {
      merchantId: merchant.id,
      name: '10次卡',
      mode: 'USAGE_BASED',
      price: 80.0,
      totalUses: 10,
      benefitsJson: { discount: 0.85, description: '享8.5折, 可分多次使用' },
      status: 'ACTIVE',
    },
  ],
});
// [DEBUG_REMOVED]

// [DEBUG_REMOVED]
// [DEBUG_REMOVED]
// [DEBUG_REMOVED]
}

main()
  .catch((e) => {
    // [DEBUG_REMOVED]
    process.exit(1);
  })
  .finally(() => prisma.$disconnect());

```